

1. Installation and Code Run

1. Following packages need to be installed before running the code

```
% pip install basis_set_exchange
% pip install numba
% pip install numba-scipy
```

2. Download the NWChem-SOC files from GitHub and put them in a folder. Set the path of nwchem in `nwchem_path` variable in the `main.py` file.

3. Make a new folder where you would like to work and add the nwchem input file for TDDFT calculation to it. To compute SOC, add the following block:

```
SOC
nsroots 3      ## Number of singlet roots of interest
ntroots 3      ## Number of triplet roots of interest
zeff 2         ## Method for Zeff calculation
nproc 8        ## Number of processors for tddft calculation
END
```

If the singlet and triplet roots are not defined, the code will assume the default of number of roots provided in the TDDFT calculation.

You should also specify the title of the file to be same as the name of the input file. For example, if my input is `soc.nw` then we should add the line title "soc". The code will search for binary files based on this title name.

4. Run the code by following command

```
% python path-to-soc-files/main.py soc.nw
```

The code will first carry out the TDDFT calculation and then compute SOC. If TDDFT calculation has already been carried out, we can comment out `nwjob.tddft_run(fname, nwchem_path)`.

2. Comments:

1. The SOC integrals are by default printed out to binary files in `soc_ab` function using `np.save`. Since this is the computationally expensive part, and if your code stops in between with an error or if you want to restart the calculation. Then do the following:

Comment out the lines:

```
start = time.time()
Vabm0, Vabm1 = soc_ab(atoms, nbasis, Z, bfn, coord)
end = time.time()
```

And Uncomment:

```
np.load('Vab0.npy', Vabm0)
```

```
np.load('Vab1.npy', Vabm1)
```